SuzzaEngine

项目源码导读

—— 从零开始读懂一个小型 Vulkan 游戏引擎 ——

适用读者	零基础 / 刚接触图形与引擎的开发者
技术栈	C++20 + Vulkan + GLFW + ImGui + glm
代码规模	约 70 个源文件、1 万行左右
文档定位	阅读地图与关键机制说明，不是 API 手册
对应代码	本仓库 main 分支

2026-10

阅读指南（先看这一页）

这份文档解决一个问题：**一个什么都不会的新手，应该按什么顺序把这一万行代码读明白。**

它不追求覆盖每一个类，而是先把“骨架”立起来：项目能干什么、怎么编译、代码分成哪几层、从哪个文件进入、一帧画面是怎么产生的。等骨架清楚了，细节可以自己按目录去查。

> **怎么用这份文档：**建议的用法：第一次只读第 1~4 章（约占全文 40%），跟着第 4 章的七站路线把代码点开看一遍；第 5 章当工具书，卡住时回来查；第 7 章的练习一定要做——不做练习的阅读，两周后基本忘光。

全文结构：

第 1 章	项目概览：它是什么、能做什么、能力边界在哪
第 2 章	把项目跑起来：环境、构建、三个自动步骤
第 3 章	全局结构地图：目录分层与模块职责
第 4 章	源码阅读路线：七站式推进
第 5 章	核心机制详解：生命周期、对象模型、渲染、反射
第 6 章	一帧的完整生命周期
第 7 章	动手练习：从改一行到加一个组件
第 8 章	常见坑与排错清单
第 9 章	术语表
附录 A	文件速查表：每个文件一句话
附录 B	关键 API 速查

第 1 章 项目概览

1.1 一句话定位

Suzza 是一个用 C++20 和 Vulkan 从零写出来的小型 3D 渲染引擎，自带一个运行时 ImGui 编辑器，用 Actor-Component 模式组织场景。

它属于“教学型 / 实验型引擎”：代码量小、没有历史包袱、每个模块都能顺着读通，非常适合理解“一个游戏引擎究竟是怎样被拆开、又怎样拼起来的”。它不适合直接拿去做商业项目——没有物理、没有动画系统、没有 UI 布局系统、没有资源热重载、没有多线程渲染。

1.2 四块核心

核心部分	做了什么	对新手的意义
Actor-Component 架构	Actor 是场景实体，Component 是可插拔能力（网格、相机、移动）；Actor 之间可父子嵌套形成变换层级	这是引擎的世界观，先懂它，后面所有代码都好理解
自研简易反射	clang 注解 + 外部 LLVM 程序自动生成类的元数据，让编辑器能在运行时按名字列出并修改字段	项目里最“魔法”的部分，也是新手最容易卡住的地方
Vulkan 渲染后端	手写实例、设备、交换链、渲染通道、管线、描述符、命令缓冲、同步对象	代码量最大，但结构清晰，可当 Vulkan 教程的配套代码读
运行时编辑器	ImGui 面板：Hierarchy 树 + Inspector 属性表，可直接改场景对象	理解反射系统价值的最佳入口

表 1-1 四大核心模块

1.3 技术栈与依赖

所有第三方库都在 `thirdparty/` 下且经过裁剪，不需要包管理器：

依赖	用途	备注
Vulkan SDK	图形 API 与 `glslc` 着色器编译器	必须安装，且要让 `find_package(Vulkan)` 成功
GLFW	窗口、输入、Surface 创建	thirdparty/glfw
glm	向量、矩阵、四元数数学库	thirdparty/glm
ImGui（docking 分支）	编辑器 UI	thirdparty/imgui-docking
cgltf	glTF 模型解析	thirdparty/cgltf，封装在 tool/modelloader

依赖	用途	备注
stb_image	图片解码（jpg/png 等）	thirdparty/stb_image，封装在 tool/imageloader

表 1-2 第三方依赖

1.4 代码规模

把 `src/`、`shared/`、`tool/` 里自己写的东西加起来，大约 70 个源文件、1 万行上下。按每天 300~500 行算，两三周可以完整过一遍。真正需要精读的只有 15 个左右的文件，第 4 章会逐个点名。

1.5 能力边界：它现在不做什么

- 没有物理引擎、没有骨骼动画、没有粒子系统。
- 没有场景序列化（关卡不能存盘），场景是在 `SuzzaApplication::Init` 里用代码搭出来的。
- 光照很简单，没有阴影、没有后处理、没有延迟渲染。
- 材质只是“一张贴图 + 一个 bindless 槽位”，没有 PBR 参数集。
- 没有资源异步加载，模型和纹理都在主线程同步读。

知道边界很重要：读代码时看到“这里怎么没有 XX”，多数不是你没找到，而是它确实没做。

第 2 章 把项目跑起来

先跑通，再读代码。 一个能编译、能出画面、能改一行就看到变化的环境，抵得上读三天源码。

2.1 准备环境

项目	要求
操作系统	Windows（项目只在 MSVC 上验证过；CMake 里有 Linux/macOS 预设，但未验证）
编译器	Visual Studio 2019/2022，需支持 C++20
构建工具	CMake 3.21 以上（预设文件用到 preset v3）、Ninja
图形 SDK	Vulkan SDK（含 glslc），建议 1.3.x
其他	Git

表 2-1 环境要求

2.2 三步构建

打开 **x64 Native Tools Command Prompt for VS**（关键：这个命令行里才有 `cl.exe` 和 Ninja），进入仓库根目录：

```
cmake --preset x64-debug
cmake --build --preset x64-debug

# 产物：out/build/x64-debug/SuzzaEngine.exe
# 直接运行它即可
```

想要 Release：把上面的 `x64-debug` 换成 `x64-release`。

> **最重要的构建约束：不要用 `cmake -G "Visual Studio 17 2022"`。** 反射代码生成器需要读取 `build/compile_commands.json`，而这个文件只有 Ninja / Makefile 这类生成器才会产出（见 `CMakeLists.txt:15` 的 `CMAKE_EXPORT_COMPILE_COMMANDS ON`）。用 VS 生成器配置后，反射那一步会因为缺少输入而失败，或者继续使用过期代码。

2.3 构建时自动做的三件事

理解这三件事，能省掉一大半“莫名其妙”的报错。

自动步骤	CMakeLists.txt 位置	产物 / 作用
编译着色器	第 67–109 行	用 `Vulkan::glslc` 把 `shaders/*.vert|frag` 编成 `<build>/shaders/*.spv`

自动步骤	CMakeLists.txt 位置	产物 / 作用
生成反射代码	第 112–135 行	运行 `tool/ReflectGenerated/` 下的生成器 exe，扫描 `src/` 输出 `*.generated.cpp`
拷贝资源	第 207–211 行	POST_BUILD 把 `assets/` 复制到 exe 同目录

表 2-2 构建期的三个自动步骤

资源路径不是靠“当前工作目录”猜的，而是由 `shared/file/FileSystem.cpp` 里的 `GetAssetsFolder()` / `GetShadersFolder()` 解析：优先用 CMake 注入的宏（`ASSETS_ROOT` / `SHADERS_ROOT`），否则退回到 **exe 所在目录**下的 `assets/`、`shaders/`。所以资源拷错位置、或者把 exe 单独挪走，都会导致“模型加载失败”。

2.4 跑通后的自检

程序起来应该看到：一个 1600×1200 的窗口，标题 `First Window`；画面里有三个层级相连的彩色立方体；有 ImGui 的 Hierarchy 与 Inspector 面板；鼠标左键拖动可以转视角，WASD 可以移动（`MovementComponent` 提供）。

如果窗口正常但画面全黑，先看控制台日志（`SUZZA_LOG_ERROR` 会打印），最常见的是 Vulkan 验证层报描述符或管线错误。

第 3 章 全局结构地图

3.1 一张图看懂目录

```
SuzzaEngine/
├─ CMakeLists.txt              顶层构建：着色器编译、反射生成、资源拷贝
├─ CMakePresets.json           x64-debug / x64-release 预设（Ninja + cl.exe）
├─ shaders/                    2 个 GLSL 文件，构建期编译成 spv
├─ assets/                     模型（gltf/bin）与纹理
│
├─ src/                        ★ 引擎与示例游戏的主体代码
│  ├─ main.cpp                 程序入口，约 37 行
│  ├─ SuzzaApplication.*       ★ 示例"游戏层"：搭场景、挂组件
│  ├─ engine/                  ★ 引擎核心
│  │  ├─ Engine.*              单例：Init / Run / Destroy，持有所有管理器
│  │  ├─ MainWindow.*          GLFW 窗口、输入、resize 回调
│  │  ├─ BaseObject.h          所有资源的公共基类（Destroy / IsAlive）
│  │  ├─ input/                InputManager：按键与鼠标状态数组
│  │  ├─ editor/               EditorUI：Hierarchy + Inspector
│  │  ├─ graphics/             Texture：VkImage + ImageView + 显存
│  │  ├─ model/                Mesh / Material / Render / 资源管理器
│  │  └─ vulkaninitialize/     Vulkan 各概念一个类 + 统一资源管理器
│  ├─ scene/                   ★ 对象模型（ECS 风格）
│  │  ├─ Scene.*               场景：Actor 注册、更新、延迟销毁、模型加载
│  │  ├─ Actor.*               实体：Transform、组件表、父子树
│  │  ├─ Component.*           组件基类 + 类型 ID 机制
│  │  ├─ components/           Mesh / Camera / Movement 三个组件
│  │  └─ actor/                PlayerController（Actor 子类）
│  ├─ reflection/              反射运行时：ClassInfo / FieldInfo / ReflectionSystem
│  ├─ surface/                 UI 抽象接口 IUIRender + ImGui 实现
│  └─ core/                    日志、数学等小工具（独立静态库）
│
├─ shared/                     跨目标共享：类型别名、文件系统、宏、数据结构
├─ tool/
│  ├─ broadcast/               自实现的单播/多播委托（C++ 信号槽）
│  ├─ imageloader/             stb_image 封装（管理器 + 调度器）
│  ├─ modelloader/             cgltf 封装
│  └─ ReflectGenerated/        ★ 反射生成器 exe + 已生成的代码
└─ thirdparty/                 imgui-docking / glfw / glm / cgltf / stb_image
```

图 3-1 目录结构（★ 为精读区域）

3.2 分层视角

比目录更重要的，是理解它的**分层**。从下往上一共四层：

层	目录	职责	依赖方向
平台 / 工具层	`shared/`、`tool/`	类型别名、文件系统、宏、委托、图片与模型 IO	无（最底层）

层	目录	职责	依赖方向
图形层	engine/vulkaninitialize/、graphics/、model/	把 Vulkan 封装成“资源对象 + 一帧绘制”	依赖工具层
对象层	scene/、reflection/	场景图、组件、元数据	依赖图形层（MeshComponent 提交绘制）
应用 / 表现层	SuzzaApplication、editor/、surface/	搭场景、出界面、跑循环	依赖以上全部

表 3-1 四层结构

一个关键分界线：Engine 只负责“跑起来”（循环、窗口、Vulkan）；SuzzaApplication 才是“游戏内容”。引擎不 new 任何具体 Actor。这条线划清了，你就知道新代码该往哪放。

3.3 模块职责速览

模块	关键类	一句话记住
引擎骨架	Engine、MainWindow、BaseObject	Engine 是唯一单例与总调度；MainWindow 管窗口与输入
场景对象	Scene、Actor、Component	Scene 拥有 Actor，Actor 拥有 Component，父 Actor 拥有子 Actor
渲染主线	Render、BufferArea	Render 编排一帧，BufferArea 录制真正跟绘制有关的命令
Vulkan 资源	VulkanInitialize、SwapChain、RenderPass、Pipeline、Descriptor、SyncObject	一个类一个 Vulkan 概念，由 VulkanResourceManager 统一持有
资源管理	ModelResourceManager、BindlessSlotsAllocator	资源用随机字符串 ID 缓存；纹理用 0~255 的槽位索引
反射	ClassInfo、FieldInfo、ReflectionSystem	类 → 字段 → getter/setter 函数对象，编辑器靠它改属性
UI	IUIRender、ImGuiUIRender、EditorUI	surface 抽象 UI 接口，editor 是具体面板
工具	ImageLoaderManager、ModelLoaderManager、Delegate	管理器 + 调度器 + 具体实现的三层结构，便于加新格式

表 3-2 模块职责

第 4 章 源码阅读路线：七站式推进

下面是一条**依赖最少、每一步都能立刻验证**的路线。不要跳站：每一站都建立在前一站的概念上。

站	目标文件	这一站要回答的问题	建议用时
1 入口	src/main.cpp	程序从哪开始？引擎和游戏谁是谁？	15 分钟
2 引擎骨架	src/engine/Engine.cpp	初始化顺序是什么？主循环每帧做了什么？	1 小时
3 游戏层	src/SuzzaApplication.cpp	一个立方体是怎么被摆到屏幕上的？	1.5 小时
4 对象模型	src/scene/	Actor / Component / 父子树怎么协作？	半天
5 渲染主线	MeshComponent + Render + BufferArea	提交队列怎么变成 draw call？	半天
6 Vulkan 资源	src/engine/vulkaninitialize/	每个 Vulkan 对象由谁创建、谁销毁？	1~2 天（可跳读）
7 反射与编辑器	src/reflection/ + editor/	Inspector 为什么能改任意字段？	半天

表 4-1 七站阅读路线

4.1 第一站：入口（src/main.cpp）

只有 37 行，但信息密度很高：

```
SuzzaApplication* suzApp = new SuzzaApplication();
suz::Engine& engine = suz::Engine::GetEngineInstance();
constexpr int WIDTH = 1600, HEIGHT = 1200;
engine.SetApplication(suzApp);
if (!engine.Init()) { /* 打印错误并返回 */ }
engine.Run();
engine.Destroy();
```

读这一站只要抓住三件事：

1. `Engine` 是单例，通过 `GetEngineInstance()` 取。
2. 引擎和游戏内容分离：引擎不知道 `SuzzaApplication` 里有什么。
3. 生命周期三件套 `Init → Run → Destroy`，后面所有资源都挂在里面。

4.2 第二站：引擎骨架（src/engine/Engine.cpp）

`Engine::Init()` 的顺序就是整个引擎的依赖顺序，务必按顺序读：

顺序	动作	为什么必须在这个位置
1	ReflectionSystem::RegisterAllReflectionClass()	编辑器与属性系统之后要查表，必须先有表
2	创建图片、模型加载调度器	之后加载资源要用
3	new MainWindow(...)	Vulkan 实例需要窗口扩展与 Surface

顺序	动作	为什么必须在这个位置
4	`m_vkResourceManager.InitVulkan()`	一次性建好实例、设备、交换链、渲染通道、管线、命令缓冲
5	`BindOnFrameBufferResizeCallback(...)`	窗口尺寸变化时重建与尺寸相关的资源
6	`new Render()`	渲染器需要上面所有 Vulkan 资源就绪
7	`m_editorUI->Init(...)`（仅 EDITOR 构建）	ImGui 需要 render pass 与 command pool 才能挂进绘制
8	`m_suzzaApplication->Init()`	**最后**才创建游戏对象——此时图形资源已全部可用

表 4-2 Engine::Init 的依赖顺序

`Engine::Run()` 的主循环只有几行骨架：

```
m_lastTime = steady_clock::now();
m_suzzaApplication->BeginPlay();
while (!m_mainWindow->WindowShouldClose()) {
    glfwPollEvents();
    m_editorUI->BeginFrame();              // 仅 EDITOR
    float dt = now - m_lastTime;           // 秒
    m_suzzaApplication->Update(dt);        // 逻辑
    m_render->Draw(dt);                    // 渲染
    m_inputManager.SetMouseOldLocation(current);
}
vkDeviceWaitIdle(device);
```

BeginPlay 只调一次： `BeginPlay` 在循环**外**调用一次，`Update` 在循环**内**每帧调用。这是 UE 风格的生命周期，组件里也有同名的 `BeginPlay / Update / Destroy` 三件套。

`Engine::Destroy()` 的顺序基本是初始化的**逆序**：应用 → 编辑器 → 模型资源 → bindless 槽位 → Vulkan 资源 → 窗口。读销毁代码是检验“你有没有真的理解谁依赖谁”的最好方式。

4.3 第三站：游戏层（src/SuzzaApplication.cpp）

141 行，是整个项目**最好的教学样例**。它完整地做了一件事：把三个立方体画到屏幕上。阅读时拆成四段：

1. **造几何体**：手写 24 个顶点（6 面 × 4 顶点，位置 + 颜色 + UV）和 36 个索引（每面 `0,1,2, 0,2,3`）。
2. **注册资源**：`new Mesh(vertices, indices)` → `LoadMesh()` 拿到字符串 ID；`new Material()` → `LoadMaterial()` 拿到材质 ID。
3. **搭场景树**：`RootActor → ChildA → GrandChild`，每个挂一个 `MeshComponent`，`CreateMesh(id)` + `CreateMaterial(id)`。
4. **挂玩家**：`new PlayerController(scene)`，再挂 `CameraComponent`，最后 `SetActorLocation({0,0,0})`。

读完这一站，你应该能回答：**“这台机器上，顶点的数据最终变成了什么样的一次 draw call？”** 如果答不出，回 5.4 节。

> **第一个练习：**第 84–85 行有一段被注释掉的 `LoadModel(... "pony_cartoon/scene.gltf")`。把它取消注释，你就从“手写立方体”切换到“读真实 glTF 模型”，是理解 `Scene::LoadModel` 的最快方式。

4.4 第四站：对象模型（src/scene/）

这一站是**架构的核心**，四个文件就够，建议按 `Component.h → Actor.h → Scene.h → 具体组件` 的顺序读。

4.4.1 Component：能力的基类

`Component` 的关键设计只有两点：

- **纯虚生命周期**：`BeginPlay()` / `Update(float)` / `Destroy()`，全由子类实现。
- **类型 ID**：模板静态计数器给每个类型分配唯一 id，配合一个宏自动实现 `GetTypeId()`。

```
template<typename T>
size_t GetStaticTypeId() { static size_t typeId = nextId++; return typeId; }

#define COMPONENT(Comp) \
  static size_t GetComponentTypeId() { return GetStaticTypeId<Comp>(); } \
  size_t GetTypeId() const override { return GetComponentTypeId(); }
```

于是 `Actor::GetActorComponent<T>()` 只需遍历组件、比较 `GetTypeId()`，就实现了“按类型取组件”，完全不需要 RTTI 与 `dynamic_cast`。

4.4.2 Actor：实体与变换层级

`Actor` 里同时存在几样东西，别混：

成员	含义
`m_components`	这个 Actor 挂载的组件列表
`m_parentActor` / `m_childActors`	父子树，用于变换继承与生命周期递归
`m_scene`	它属于哪个场景（无场景的 Actor 不会被更新）
`m_status`	`Active` / `PendingDestroy` / `Destroyed`，支撑延迟销毁
`m_location` / `m_rotation` / `m_scale`	本地变换；旋转内部是 `glm::quat`，对外提供欧拉角接口

表 4-3 Actor 的关键成员

变换的核心就是矩阵乘法链：

```
本地矩阵 = translate(m_location) * toMat4(m_rotation) * scale(m_scale)
世界矩阵 = 父世界矩阵 * 本地矩阵
模型矩阵 = 世界矩阵                          // 直接传给 shader
```

`SetActorWorldLocation()` 的实现值得单独看一眼：它用**父世界矩阵的逆**去乘目标世界坐标，反推出本地坐标。这正是“世界坐标与本地坐标互转”的标准做法。

4.4.3 Scene：容器与调度者

`Scene` 把“所有权”和“结构”分开管理：`vector<unique_ptr<Actor>>` 拥有全部对象，`vector<Actor*>` 只存根节点。三个行为必须理解：

- **只更新根节点**：`Scene::Update` 只遍历根节点列表，子节点由父节点的 `Update` 递归调用。这样保证父在子之前更新。
- **延迟销毁**：`MarkedActorDestroy` 只把状态改成 `PendingDestroy`。本帧仍走完更新，帧末统一 `Destroy()`，下一帧才从数组里 erase——避免“正在遍历时删元素”。
- **BeginPlay 分两遍**：先对所有 Actor 批量注册子组件，再对根节点调 `BeginPlay`——保证组件被使用前都已挂到 Actor 上。

> **值得抄写的一段代码：**`Scene::LoadModel()` 是一个“从文件到场景”的完整流水线样例：遍历 `ModelData` → 转换顶点结构 → 建 Mesh/Material → 建 Actor → 注册 → 挂组件。以后要支持新格式，照着这个函数的结构抄即可。

4.5 第五站：渲染主线

这是最容易迷路的一站，但只要抓住**一个队列**和**一次录制**：

```
MeshComponent::BeginPlay()                      // 资源就绪时执行一次
  └ 组装 ModelConstant{ modelMatrix, slotIndex }
     └ Render::SubmitMesh(mesh, constant)       // 塞进 m_meshQueue

Render::Draw(dt)                                // 每帧
  ├ vkWaitForFences / vkAcquireNextImageKHR     // 等上一帧、取图像
  ├ BufferArea::RecordCommandBuffer(cmd, ...)   // 录制
  │    ├ vkCmdBeginRenderPass
  │    ├ for each mesh: BindVertexBuffers / BindIndexBuffer / DrawIndexed
  │    └ m_engineUIDelegate.Execute(cmd)        // ImGui 画在这里
  ├ vkQueueSubmit
  └ vkQueuePresentKHR
```

关键理解：`MeshComponent` 不直接画。它只把“要画什么 + 用什么矩阵”提交进队列，由 `Render` 每帧统一录制。这种“提交 + 统一绘制”是所有引擎的常规做法，好处是渲染顺序、状态排序都能集中控制。

4.5.1 BufferArea 里真正干活的三行

`RecordCommandBuffer` 内部是纯粹的 Vulkan 代码，新手只需定位这几行：

行号（约）	调用	作用
228	`vkCmdBeginRenderPass(...)`	开始渲染通道，之后所有绘制都落在帧缓冲上

行号（约）	调用	作用
231	`vkCmdBindPipeline(...)`	绑定图形管线（着色器 + 各种图形状态）
255	`vkCmdBindVertexBuffers(...)`	绑定当前 mesh 的顶点缓冲
258	`vkCmdBindIndexBuffer(..., VK_INDEX_TYPE_UINT32)`	绑定索引缓冲，索引是 32 位
267	`vkCmdDrawIndexed(indexCount, 1, 0, 0, 0)`	**真正的绘制调用**
272	`m_engineUIDelegate.Execute(cmd)`	ImGui 的绘制命令插在同一通道末尾

表 4-4 录制命令时的关键调用

4.6 第六站：Vulkan 资源层（可跳读）

`vulkaninitialize/` 的组织方式是“**一个类 = 一个 Vulkan 概念**”，所以可以按需查表，不必线性读完：

类	对应的 Vulkan 概念	补充说明
`VulkanInitialize`	Instance / Surface / 物理设备 / 逻辑设备 / 队列	还处理验证层与 `DescriptorIndexing` 特性链（bindless 的前提）
`SwapChain`	交换链、图像视图	窗口 resize 时重建
`RenderPass`	渲染通道（颜色 + 深度附件）	决定帧缓冲格式
`Pipeline`	图形管线	读取 `shaders/` 下的 spv
`Descriptor`	描述符池 / 集 / 布局	相机 UBO + bindless 纹理数组
`BufferArea`	顶点/索引/UBO、帧缓冲、命令池、命令缓冲	**名字容易误解**：它其实是“与交换链尺寸绑定的整套资源”
`SyncObject`	信号量 + 围栏	`MAX_FRAMES_IN_FLIGHT = 2`
`VulkanResourceManager`	以上全部的持有者	`InitVulkan()` 按依赖顺序一次性建好

表 4-5 Vulkan 层的类与概念对应

> **不要陷进结构体细节：**读这一站的正确姿势：**先建立“有哪些对象、谁创建谁销毁”的地图，不要试图背熟每个 `VkCreateInfo` 的字段。**需要写代码时再回来查。

4.7 第七站：反射与编辑器

理解这一站，你就明白“为什么 Inspector 能改任意字段”。四步链路：

1. **写标记**：类里写 `SUZZA_REFLECT_CLASS()`，字段上写 `SUZZA_PROPERTY(Editability::Editable)`。
2. **变注解**：在 clang 环境下宏展开成 `__attribute__((annotate("property:...")))`。注意——**在 MSVC 下它展开为空**，纯粹是给外部解析工具看的标记。

3. **生成元数据**：外部 LLVM 工具扫描源码，生成 `Actor.generated.h/.cpp` 等文件，内容是 `ClassInfo{ 类名，类路径，字段列表 }`，每个 `FieldInfo` 带 `getterFunc` / `setterFunc` 与字段偏移。
4. **被消费**：`EditorUI::DrawInspectorPanel` 遍历 `GetClassInfo().fields`，按 `typeName` 分派控件（如 `glm::vec3`、`float`），调 getter 取值、画控件、setter 写回。

所以真正的“魔法”不在运行时，而在**生成阶段**：反射信息是代码生成产物，不是运行时探测出来的。

> **最容易被这个坑到：**`SUZZA_REFLECT_CLASS()` 用 `__LINE__` 拼出标识符（形如 `Actor_REFLECT_CLASS_30_REFLECT_BODY`）。**只要你在这个类里增删了行，就得重新生成反射代码**，否则编译报“找不到该标识符”。详见 5.6 节。

第 5 章 核心机制详解

本章是工具书：卡住时回来查对应小节即可，不必顺序阅读。

5.1 引擎生命周期

阶段	做什么	不该做什么
`Init()`	按依赖顺序创建全局资源、窗口、Vulkan、渲染器、编辑器，最后调应用 Init	不要在 Init 里跑循环或提交绘制
`Run()`	循环：轮询事件 → UI 新帧 → 算 dt → 逻辑更新 → 渲染 → 更新输入快照	不要在 Run 里 new 全局资源
`Destroy()`	逆序释放，最后 `vkDeviceWaitIdle` 保证 GPU 空闲	不要遗漏 ImGui 与槽位分配器的销毁

表 5-1 三个阶段的职责边界

5.2 对象模型：三个必须记住的规则

- **所有权**：Scene 用 `unique_ptr` 拥有 Actor；Actor 用裸指针 + `new / delete` 管组件（在 `Destroy()` 里逐个删）。风格不统一，看代码时注意“谁负责 delete”。
- **更新顺序**：父 → 子；组件先于子 Actor 更新。所以父的变换在本帧已是最新，子节点拿到的父世界矩阵是一致的。
- **延迟销毁**：任何“在遍历中删对象”的需求，都通过 `MarkedActorDestroy` + 帧末统一处理实现。

为什么到处能看到 ValidPtr： `Component` 里有 `shared_ptr<bool> m_isAlive`，`GetValidPtr()` 把它交出去。这样对象被销毁后，外部还能安全判断“它是不是还活着”，避免悬空指针。`Material` 的析构会检查它，并在未调用 `Destroy` 时打印警告。

5.3 资源管理：随机 ID + bindless 槽位

机制	解决的问题	关键 API
`ModelResourceManager`	同一份 Mesh / Material 不重复上传显存；组件之间用字符串 ID 互相引用	`LoadMesh / LoadMaterial` 返回 id，`GetLoadedMesh(id)` 取回
`BindlessSlotsAllocator`	把多张纹理塞进一个描述符数组，shader 里用下标采样	`AllocateBindlessSlots` / `MarkedTargetSlotsRelease` / `ReleaseSlots`

表 5-2 两层资源缓存

为什么用随机字符串而不是自增 id？随机 id 可以在“卸载后重新加载”时避免与旧引用混淆，跨模块传递也不需要同步计数器。代价是可读性差——调试时建议在 `LoadMesh` 上打断点，看返回值。

纹理槽位上限由 `shared/macros/VkMacro.h` 里的 `MAX_SAMPLED_TEXTURE_IMAGES_SLOTS`（256）决定，`MAX_FRAMES_IN_FLIGHT`（2）也在同一文件。

5.4 渲染管线：从提交到上屏

步骤	API	出错时怎么处理
等上一帧	`vkWaitForFences`	阻塞等待，无失败分支
取图像	`vkAcquireNextImageKHR`	`VK_ERROR_OUT_OF_DATE_KHR` → 重建交换链并跳过本帧
录制	`vkResetCommandBuffer` + `RecordCommandBuffer`	纯 CPU 端记录命令
提交	`vkQueueSubmit`（等 imageAvailable，signal renderFinished）	失败抛异常
呈现	`vkQueuePresentKHR`	`OUT_OF_DATE` / `SUBOPTIMAL` → 重建交换链；其余失败抛异常
帧号推进	`m_currentFrame = (m_currentFrame + 1) % 2`	与双缓冲帧资源对应

表 5-3 Render::Draw 的六个步骤

为什么是 2 帧在飞？ 单帧意味着 CPU 必须等 GPU 画完才能准备下一帧；2 帧可以让 CPU 准备第 N+1 帧时 GPU 还在画第 N 帧。这也是为什么相机 UBO、命令缓冲、信号量都是长度 2 的数组。

两个下标别混： `m_currentFrame` 与交换链的 `imageIndex` 不是一回事：前者是“帧资源下标”（固定 2 个），后者是“当前画到哪张交换链图像”（通常 3 张）。混用会出同步错误——这是新手读这段代码最常见的误解。

5.5 Vulkan 对象地图：谁创建、谁销毁

对象	创建位置	销毁位置
`VkInstance` / DebugMessenger / Surface	`VulkanInitialize` 的初始化流程	`VulkanInitialize::Destroy`
`VkDevice` / 队列	`CreateLogicalDevice`	同上
`VkSwapchainKHR` + ImageViews	`SwapChain::InitSwapChain`	`SwapChain::CleanUpSwapChain` + 重建
`VkRenderPass` + 帧缓冲	`RenderPass` / `BufferArea::CreateFrameBuffers`	各自 `Destroy` / `CleanupBufferArea`
`VkPipeline` / `VkPipelineLayout`	`Pipeline`	`Pipeline::Destroy`
`VkDescriptorPool/Set`	`Descriptor`	`Descriptor::Destroy`

对象	创建位置	销毁位置
VkBuffer + VkDeviceMemory	BufferArea::CreateVertexBuffer 等	CleanupBufferArea
VkImage（纹理 / 深度）	Texture / CreateDepthResources	各自 Destroy
信号量 / 围栏	SyncObject	SyncObject::Destroy

表 5-4 Vulkan 对象的创建与销毁归属

5.6 反射系统：全链路与三个陷阱

5.6.1 数据结构

类型	作用	关键点
FieldInfo	描述一个字段	名字、类型名、所属类名、**字节偏移 offset**、可编辑性、getter / setter 函数对象
ClassInfo	描述一个类	类名、源文件路径、父类名列表、字段列表
ReflectionSystem	全局注册表	RegisterClassInfo 写入 map，之后按类名查
ContainerAccessor	容器字段读写	为 Array 等类型提供统一的读写接口

表 5-5 反射的四个运行时类型

字段偏移用 SUZZA_OFFSET_OF(Actor::m_location) 计算——本质是“把空指针 0 强转成类指针后取成员地址”，得到的就是偏移量。getter / setter 则是编译期生成的 lambda，所以运行时几乎没有反射开销。

5.6.2 三个陷阱

1. **行号敏感**：宏把 __LINE__ 拼进标识符（如 Actor_REFLECT_CLASS_30_REFLECT_BODY）。你在这行之前增删任何行，生成的头文件就对不上，编译报“未定义的标识符”。
2. **依赖外部工具**：生成器是仓库里预编译的 exe，依赖 LLVM 与 compile_commands.json。换机器、换编译器、换构建目录都可能需要重新配置。
3. **生成文件写死了绝对路径**：Actor.generated.cpp 里的 classPath 是形如 D:/LjhUser/.../src/scene/Actor.h 的字符串。换目录后该字段失效（目前只影响编辑器显示，不影响编译）。

新手绕坑方案：只想给已有反射类**加一个新字段**时，最省事的路径是：照抄现有 FieldInfo::CreateFieldInfo(...) 那行的写法，在生成文件里手写一行，并同步生成头文件里的结构；等 LLVM 环境就绪再正规重生成。

5.7 编辑器、UI 与输入

- **UI 抽象**：`src/surface/UIRender.h` 定义 `IUIRender` 接口，`ImGuiUIRender` 是它的实现。换 UI 库只需换实现，不动编辑器逻辑。
- **编辑器接线**：`EditorUI::Init` 把绘制函数绑定到 `EngineUIRenderEventDelegate`；`BufferArea` 在录制末尾调 `m_engineUIDelegate.Execute(cmd)`，于是 ImGui 的命令落在同一个渲染通道内。
- **Hierarchy**：遍历场景根节点画树，`TreeNodeEx` + 点击选中 + 右键菜单删除 Actor。
- **Inspector**：遍历选中 Actor 的 `ClassInfo.fields`，按 `typeName` 分派控件（`glm::vec3` → 三个 float 拖条；`float` → 单值拖条），改完调 setter 写回。
- **输入**：`InputManager` 用两个定长数组（按键 256、鼠标 16）记录状态，`MainWindow` 是它的 friend，在 GLFW 回调里写入。每帧末尾把“本帧鼠标位置”存为“上一帧位置”，用于算增量。

第 6 章 一帧的完整生命周期

把前面所有章节串起来，一帧发生的事情如下（括号内是发生位置）：

```
glfwPollEvents()                                    (Engine::Run)
EditorUI::BeginFrame()                              (ImGui 新帧)
deltaTime = now - lastTime

SuzzaApplication::Update(dt)                        (游戏逻辑)
  └ Scene::Update
       └ 仅根 Actor → Actor::Update 递归子 Actor
            └ 每个 Component::Update
                 ├ MovementComponent: 读输入 → 改 Transform
                 └ MeshComponent:      （当前为空）

Render::Draw(dt)                                    (渲染)
  ├ vkWaitForFences
  ├ vkAcquireNextImageKHR        → 需要时重建交换链
  ├ 更新相机 UBO（当前帧槽位）
  ├ BufferArea::RecordCommandBuffer
  │    ├ BeginRenderPass
  │    ├ 遍历 m_meshQueue → Bind + DrawIndexed
  │    ├ ImGui 命令（Hierarchy / Inspector）
  │    └ EndRenderPass
  ├ vkQueueSubmit（等 imageAvailable，signal renderFinished）
  ├ vkQueuePresentKHR
  └ m_currentFrame = (m_currentFrame + 1) % 2

InputManager: current → old                       (帧末快照)
```

图 6-1 一帧的时序

用四个问题自检：

1. 为什么 `Scene::Update` 里只对根 Actor 调 `Update`？（答：父先子后，且避免重复更新）
2. 为什么 Mesh 要在 `BeginPlay` 提交，而不是每帧提交？（答：入队是“资源就绪”的一次性动作；之后变换变化只需更新矩阵）
3. `m_currentFrame` 为什么对 2 取模？（答：双缓冲帧资源，防止 CPU 覆盖 GPU 正读的数据）
4. 窗口拉伸时哪些东西必须重建？（答：交换链 + 图像视图 + 帧缓冲 + 深度资源，它们都与 extent 绑定）

第 7 章 动手练习

读十遍不如改一遍。 下面按难度排序，每项都给了涉及文件与验收标准。

难度	任务	涉及文件	验收标准
★	改窗口标题和初始尺寸	`src/main.cpp`	标题变成你想要的名字，尺寸变化后画面仍正常
★	移动 / 缩放立方体，观察父子联动	`src/SuzzaApplication.cpp`	移动 RootActor 时子节点跟着走；只动 ChildA 时 GrandChild 跟随
★★	取消注释模型加载，载入 `pony_cartoon`	`SuzzaApplication.cpp` 第 84–85 行	能看到完整的马与车，贴图正确
★★	写一个 `RotateComponent`：每帧绕 Y 轴转一点	新建 `scene/components/RotateComponent.*`，参照 `MovementComponent`	立方体持续旋转，不依赖输入
★★★	让 Inspector 显示并修改你新加的 `float` 字段	`src/reflection/`、`engine/editor/EditorUI.cpp`	面板里出现该字段，拖动后画面立刻变化
★★★	改片元着色器，让颜色随 UV 变化	`shaders/simple_fragmentshader.frag`	重新构建后表面出现渐变或棋盘
★★★★	实现相机跟随：相机位置 = 目标位置 + 偏移	`CameraComponent`、`Actor::GetActorWorldLocation`	移动目标物体时视角平滑跟随
★★★★★	给 Scene 加“按名字查找并销毁 Actor”	`src/scene/Scene.*`	能按字符串找到并删除指定 Actor，且不崩（延迟销毁生效）

表 7-1 分级练习

7.1 加一个新组件的标准步骤

以 `RotateComponent` 为例，完整流程如下（这也是你以后扩展引擎的通用套路）：

1. 在 `src/scene/components/` 新建 `RotateComponent.h`，继承 `Component`，写上 `COMPONENT(RotateComponent)` 宏。
2. 实现 `BeginPlay / Update / Destroy`，在 `Update` 里调 `m_actorOwner->SetActorRotation(...)`。
3. 在 `SuzzaApplication::Init` 里 `actor->RegisterComponent<RotateComponent>()`。
4. 重新 configure + build。

> **别忘重新 configure：** `src/scene/components/` 下的 cpp 是靠 `GLOB_RECURSE` 收集的，**新增文件后必须重新执行 `cmake --preset`**，否则链接期会报符号未定义。

第 8 章 常见坑与排错清单

8.1 构建期

现象	原因	处理
`compile_commands.json` 找不到	用了 VS 生成器，或没先 configure	改用 Ninja 预设，先 `cmake --preset`
反射生成失败 / 生成代码过期	生成器 exe 缺失或 LLVM 环境问题	确认 `tool/ReflectGenerated/` 下的 exe 存在；必要时手动维护生成文件
`glslc` 找不到	Vulkan SDK 未安装或不在 PATH	安装 SDK，确认 `find_package(Vulkan)` 成功
中文注释乱码 / C4819	源码是 **GBK** 编码，编辑器按 UTF-8 打开	编辑器设 `files.encoding: gbk`；新增文件保持与仓库一致
新增 cpp 后链接报未定义	`GLOB` 未刷新	重新执行 `cmake --preset`
奇怪的增量编译错误	旧的构建缓存	删掉 `out/build/x64-debug` 重新配置

表 8-1 构建期常见问题

8.2 运行期

现象	最可能的原因
窗口起来了但全黑	验证层报描述符 / 管线错误；或相机矩阵异常导致物体在视锥外
模型加载失败 / 找不到文件	`assets/` 没拷到 exe 旁边；或 `GetAssetsFolder()` 解析到了别的路径
纹理全黑或全白	bindless 槽位未分配 / 未释放；或 UV 与采样器配置不对
拉伸窗口后崩溃或花屏	交换链相关资源未随 extent 重建
ImGui 面板看不见	绘制回调未绑定，或录制时没插到 render pass 内
改字段后没反应	字段不是 `SUZZA_PROPERTY`，或改了但没触发重绘

表 8-2 运行期常见问题

8.3 三个“文档级”遗留问题

- `README_zh.md` 第 14–18 行、第 50–57 行仍残留 Git 冲突标记（`<<<<<<< HEAD` 等），阅读时跳过这几段。
- 生成的反射文件里写死了作者机器的绝对路径（见 5.6.2）。
- `complieShader.bat` 硬编码了作者本机的 Vulkan SDK 路径，只能作为“手动编译着色器”的参考。

第 9 章 术语表

术语	在本项目中的含义
Actor	场景中的一个实体，有名字、变换、组件列表，可挂父子关系
Component	挂在 Actor 上的能力单元，实现 `BeginPlay / Update / Destroy`
Scene	Actor 的容器与世界，负责更新、销毁、模型加载
BeginPlay	对象进入运行状态时的初始化钩子，只调一次
延迟销毁	遍历期间不真正删除，只标记 `PendingDestroy`，帧末统一销毁
bindless	把多张纹理放进一个描述符数组，shader 用下标访问
槽位（slot）	bindless 数组中的下标，本项目上限 256
frames in flight	允许 CPU 提前准备的帧数，本项目为 2
交换链	一组用于呈现的屏幕图像，需要按窗口尺寸重建
渲染通道	一次绘制所依赖的附件与子流程描述
反射	运行时可查询类的字段信息；本项目通过代码生成实现
ClassInfo / FieldInfo	反射的运行时数据结构，分别描述类与字段
ModelConstant	传给 shader 的每物体常量：模型矩阵 + 纹理槽位
管线（Pipeline）	管线状态对象，固定了着色器与各种图形状态

表 9-1 术语对照

附录 A 文件速查表

按目录列出主要文件及一句话职责，方便“我知道功能，但不知道在哪个文件”。

A.1 引擎核心（src/engine/）

文件	职责
Engine.h/.cpp	单例与总调度：Init / Run / Destroy，持有窗口、Vulkan、渲染器、编辑器
MainWindow.h/.cpp	GLFW 窗口、输入回调、resize 委托、Surface 创建
BaseObject.h	所有资源的公共基类：Destroy() + IsAlive / GetValidPtr
input/InputManager.h	按键与鼠标状态数组、鼠标位置快照
editor/EditorUI.h/.cpp	Hierarchy 树 + Inspector 属性表
graphics/Texture.h/.cpp	VkImage + ImageView + 显存分配 + 槽位更新
model/Mesh.h/.cpp	顶点 / 索引缓冲与顶点布局描述
model/Material.h/.cpp	材质 = 一张纹理 + 一个 bindless 槽位
model/Render.h/.cpp	一帧的编排：取图、录制、提交、呈现；持有 mesh 提交队列
model/ModelResourceManager.h	Mesh / Material 的 ID 缓存与反向查找
model/BindlessSlotsAllocator.h	纹理槽位分配与延迟回收

A.2 Vulkan 层（src/engine/vulkaninitialize/）

文件	职责
VulkanInitialize.h/.cpp	实例、Surface、物理 / 逻辑设备、队列、验证层、特性链
SwapChain.h/.cpp	交换链、图像视图、重建
RenderPass.h/.cpp	渲染通道（颜色 + 深度）
Pipeline.h/.cpp	图形管线与布局
Descriptor.h/.cpp	描述符池 / 布局 / 集（相机 + bindless 纹理）
BufferArea.h/.cpp	顶点 / 索引 / UBO、帧缓冲、命令池 / 缓冲、录制命令
SyncObject.h/.cpp	信号量与围栏
VulkanResourceManager.h/.cpp	以上对象的统一持有与初始化顺序

A.3 场景与反射（src/scene/、src/reflection/）

文件	职责
scene/Scene.h/.cpp	Actor 注册、根节点更新、延迟销毁、模型加载流水线
scene/Actor.h/.cpp	实体：变换、组件表、父子树、按类型取组件
scene/Component.h	组件基类、类型 ID 机制、存活标记

文件	职责
scene/components/MeshComponent.*	把 Mesh + Material + 模型矩阵提交给 Render
scene/components/CameraComponent.*	视图 / 投影矩阵、FOV、裁剪面
scene/components/MovementComponent.*	鼠标转视角 + WASD 移动
scene/actor/PlayerController.*	挂了 MovementComponent 的 Actor 子类
reflection/ReflectionInfo.h/.cpp	ClassInfo / FieldInfo / ReflectionSystem
reflection/ContainerAccessor.h	容器字段的统一读写接口

A.4 共享与工具（shared/、tool/）

文件	职责
shared/SuzzaTypes.h	suz::uint32 等类型别名
shared/file/FileSystem.cpp	exe 目录、源码目录、shaders、assets 的路径解析
shared/macros/VkMacro.h	MAX_FRAMES_IN_FLIGHT、纹理槽位上限等常量
shared/macros/ReflectMacros.h	反射相关的全部宏
tool/broadcast/	单播 / 多播委托（信号槽），用于 resize 回调与 UI 绘制回调
tool/imageloader/	图片加载：管理器 → 调度器 → stb_image 实现
tool/modelloader/	模型加载：管理器 → 调度器 → cgltf 实现
tool/ReflectGenerated/	反射生成器 exe 与已生成的元数据代码
src/core/debug/SuzzaLog.h	日志宏，生产构建下编译为空

附录 B 关键 API 速查

B.1 引擎与场景

```
suz::Engine& e = suz::Engine::GetEngineInstance();
e.SetApplication(app);  e.Init();  e.Run();  e.Destroy();

auto* window   = suz::Engine::GetEngineInstance().GetMainWindow();
auto* render   = suz::Engine::GetEngineInstance().GetRender();
auto& modelMgr = suz::Engine::GetModelResourceManager();

scene->RegisterActor(actor);
scene->RegisterActor(child, parent, /*bRegisterComponents=*/false);
scene->MarkedActorDestroy(actor);      // 延迟销毁
```

B.2 Actor 与组件

```
actor->RegisterComponent<MeshComponent>();
auto* mesh = actor->GetActorComponent<MeshComponent>();
auto  all  = actor->GetAllActorComponents();

actor->SetActorLocation(glm::vec3(1, 0, 0));          // 本地坐标
actor->SetActorWorldLocation(glm::vec3(3, 1.2f, 0));  // 世界坐标
glm::mat4 m = actor->GetActorWorldTransform();
actor->GetActorForwardVector();   // 同样有 Right / Up
```

B.3 渲染提交

```
auto meshId = modelMgr.LoadMesh(vertices, indices);   // 或 LoadMesh(Mesh*)
auto matId  = modelMgr.LoadMaterial(new suz::Material(texturePath));

auto* mc = new suz::MeshComponent();
mc->CreateMesh(meshId);
mc->CreateMaterial(matId);
actor->RegisterComponent<suz::MeshComponent>();
```

B.4 反射标记

```
class MyThing : public suz::Actor {
  SUZZA_REFLECT_CLASS()                // 位置敏感：行号会进宏名
public:
  SUZZA_PROPERTY(suz::Editability::Editable)
  float m_myFloat = 1.0f;
};
```

— 文档结束 —